

Module : DataOps et MLOps

Observatoire Intelligent des Prix Immobiliers

Rapport de Projet

Gestion Agile – Qualité des Données & Data Contracts –
Industrialisation, Déploiement et Monitoring

Product Owner	Nohaila Ichou
Scrum Master	Moussaoui Abdelkarim
Data Engineer 1	Hajar El Khalidi
Data Engineer 2	Zainab Rhouni
Data Quality Engineer	Douaa Bounadar
ML Engineer 1	Salama Boussaid
ML Engineer 2	Malika Chaaban
DevOps / Deployment Engineer	Youness Zouhairi

Encadrant : Pr. Mohammed Ait Daoud

Dépôt : `hajar-elkhalidi/mlops_meia_v2`

Année universitaire 2025–2026

Contents

Avant-propos	5
I Gestion Agile du Projet	6
1 Introduction et organisation de l'équipe	7
1.1 Introduction	7
1.2 Organisation de l'équipe	7
1.3 Mise en place du projet dans Jira	8
2 Structure Agile du projet	9
2.1 Hiérarchie Jira	9
2.2 Les quatre Epics	9
2.3 Product Backlog	10
3 Déroulement détaillé des sprints	12
3.1 Sprint 1 – Cadrage, Data Strategy et Ingestion	12
3.2 Sprint 2 – Transformation, Qualité et Orchestration	15
3.3 Sprint 3 – Machine Learning et MLflow	16
3.4 Sprint 4 – FastAPI, Docker, CI/CD et Monitoring	17
3.5 Tasks spécifiques au Scrum Master	18
3.6 Planning global des sprints	19
4 Cérémonies, qualité de processus et suivi	20
4.1 Workflow du Board Jira	20
4.2 Cérémonies Scrum	20
4.3 Definition of Ready	21
4.4 Definition of Done	21
4.5 Sprint Reviews	21
4.6 Sprint Retrospectives	23
4.7 Registre des obstacles	24
4.8 Gestion des risques	25
4.9 Suivi de l'avancement	25
4.10 Conclusion de la partie Agile	26
II Qualité des Données & Data Contracts	27
5 Stratégie de qualité des données	28
5.1 Introduction et enjeux	28

6	Modélisation dbt en couches : Staging, Intermediate, Marts	29
6.1	Contexte et objectif de la mission	29
6.2	Architecture en couches (Medallion Architecture)	29
6.3	Tests de qualité génériques (17 tests)	29
6.4	Documentation, lignage et validation concrète	30
6.5	Conclusion	30
7	Data Contracts et tests de qualité	31
7.1	Tests génériques (structure & intégrité) – rappel	31
7.2	Tests customisés SQL (validité & cohérence métier)	31
8	Documentation, dictionnaire de données et lineage	33
8.1	Documentation et dictionnaire de données	33
8.2	Data Lineage (graphe de dépendances)	34
9	Conclusion et passation vers le MLOps	35
III	Industrialisation, Déploiement et Monitoring	36
10	Résumé exécutif et périmètre	37
10.1	Résumé exécutif	37
10.2	Objectif du travail et périmètre de contribution	37
11	Architecture globale de la solution finale	39
11.1	Chaîne fonctionnelle DataOps / MLOps	39
11.2	Services du déploiement final	39
12	Modélisation Machine Learning : préparation des données et comparaison des modèles	40
12.1	Préparation des données	40
12.2	Modèles entraînés	40
12.3	Évaluation et comparaison des modèles	40
12.4	Modèle retenu	40
13	Tracking des expérimentations et Model Registry avec MLflow	42
13.1	Mise en place du tracking MLflow	42
13.2	Model Registry	42
13.3	Détection de dérive (monitoring)	42
14	Reconstruction locale : démarche chronologique	43
14.1	Reprise du dépôt et compréhension de l'état initial	43
14.2	Reconstruction locale de la stack Docker	43
14.3	Vérification de l'ingestion et de la base DuckDB	44
14.4	Validation des transformations dbt	45
14.5	Validation de l'API et des tests automatiques	45
14.6	Mise en place et validation de la CI	45
15	Industrialisation technique : unification de la chaîne	47
15.1	Correction des chemins de données et des variables d'environnement	47
15.2	Intégration MLflow et gestion du modèle	47
15.3	Orchestration Dagster de bout en bout	47

16 Déploiement sur Komodo	48
16.1 Compréhension du comportement de la stack dans Komodo	48
16.2 Validation inter-services dans le conteneur Dagster	48
16.3 Exécution complète du pipeline sur le serveur	48
17 Ajout du monitoring persistant	50
17.1 Pourquoi ajouter un service de monitoring dédié ?	50
17.2 Validation d'une prédiction après intégration du monitoring	50
17.3 Test de persistance après redémarrage de l'API	50
17.4 Validation de la dérive avec garde-fou sur le nombre d'échantillons	50
18 Erreurs rencontrées et solutions appliquées	52
19 Résultats finaux et synthèse de conformité	54
19.1 Résultats fonctionnels	54
19.2 Synthèse de conformité	54
19.3 Bonnes pratiques retenues	54
Conclusion générale	56

List of Figures

2.1	Les quatre Epics configurés dans Jira	10
2.2	Vue générale du Product Backlog dans Jira	10
3.1	Sprint Backlog du Sprint 1 dans Jira	13
4.1	Board Jira pendant le Sprint 1	20
7.1	Test SQL <code>assert_calcul_prix_m2.sql</code>	32
7.2	Test SQL <code>assert_surface_logique.sql</code>	32
8.1	Dictionnaire de données dbt : colonnes et types	34
8.2	Graphe de Data Lineage (DAG dbt)	34
14.1	Analyse des écarts entre branche locale et <code>origin/main</code>	43
14.2	Exécution de <code>docker compose up -d</code>	44
14.3	Contrôle de la base DuckDB (a)	44
14.4	Contrôle de la base DuckDB (b)	45
14.5	Exécution de <code>dbt run</code>	45
14.6	Pipeline CI GitHub Actions	46
16.1	Services Komodo à l'état DOWN	48
16.2	Services Komodo à l'état RUNNING	49
16.3	Exécution réussie du pipeline Dagster	49
17.1	Test de prédiction après intégration du monitoring	50
17.2	Monitoring de la dérive (drift) avec garde-fou	51

Avant-propos

Ce rapport retrace l'intégralité du cycle de vie du projet **Observatoire Intelligent des Prix Immobiliers**, réalisé dans le cadre du module DataOps et MLOps, depuis la gestion Agile de l'équipe jusqu'au déploiement et au monitoring du service de prédiction en production, en passant par la garantie de la qualité des données.

Le rapport est structuré en trois grandes parties, correspondant chacune à un livrable et à une étape clé de la chaîne DataOps / MLOps :

- **Partie I. Gestion Agile du Projet** – organisation Scrum de l'équipe, structuration du Product Backlog dans Jira, planification et exécution détaillée des quatre sprints, cérémonies, Definition of Ready / Definition of Done, Sprint Reviews, Sprint Retrospectives et gestion des risques et obstacles.
- **Partie II. Qualité des Données & Data Contracts** – mise en place des tests dbt, du Data Contract et du Data Lineage garantissant un dataset certifié en amont de la modélisation.
- **Partie III. Industrialisation, Déploiement et Monitoring** – reconstruction de la chaîne technique, orchestration Dagster, conteneurisation Docker, intégration continue, déploiement sur Komodo et mise en place d'un monitoring persistant (Rapport final DevOps).

Logique générale du projet : Collecter → Fiabiliser → Apprendre → Industrialiser.

Ces trois parties sont volontairement présentées dans cet ordre car elles reflètent la progression réelle du projet : l'organisation Agile a structuré le travail de l'équipe sur quatre sprints (Partie I) ; à l'intérieur du Sprint 2, la Data Quality Engineer a mis en place les contrôles garantissant qu'aucune donnée non conforme ne puisse atteindre les ML Engineers (Partie II) ; enfin, le DevOps Engineer a relié l'ensemble des briques produites par l'équipe pour livrer un service déployé, observable et reproductible (Partie III)

Part I

Gestion Agile du Projet

CHAPTER 1

Introduction et organisation de l'équipe

1.1 Introduction

Le développement du projet Observatoire Intelligent des Prix Immobiliers est organisé selon une démarche Agile inspirée du framework Scrum. Cette organisation permet de structurer le travail de l'équipe, de répartir les responsabilités et de construire progressivement la chaîne DataOps / MLOps.

Le projet est organisé autour de quatre sprints :

- Sprint 1 Cadrage, Data Strategy et Ingestion
- Sprint 2 Transformation, Qualité et Orchestration
- Sprint 3 Machine Learning et MLflow
- Sprint 4 Déploiement et Monitoring

La gestion du projet repose notamment sur des Epics, un Product Backlog, des User Stories avec leurs critères d'acceptation, des Subtasks techniques, des Tasks spécifiques au Scrum Master, un Sprint Backlog, des Sprint Reviews et Retrospectives, une Definition of Ready, une Definition of Done, ainsi qu'un suivi des risques et des obstacles.

1.2 Organisation de l'équipe

#	Rôle	Membre	Responsabilité principale
1	Product Owner	Nohaila Ichou	Vision produit, besoins métier, gestion du Product Backlog, priorisation et validation fonctionnelle
2	Scrum Master	Moussaoui Abdelkarim	Organisation Scrum, planification, suivi des sprints, coordination et gestion des obstacles
3	Data Engineer 1	Hajar El Khalidi	Collecte, ingestion et orchestration des données
4	Data Engineer 2	Zainab Rhouni	Stockage, transformation et structuration des données
5	Data Quality Engineer	Douaa Bounadar	Contrôle qualité, règles de validation, tests des données
6	ML Engineer 1	Salama Boussaid	Feature engineering, entraînement, reproductibilité ML
7	ML Engineer 2	Malika Chaaban	Expérimentation, comparaison des modèles, MLflow

#	Rôle	Membre	Responsabilité principale
8	DevOps / Deployment Engineer	Youness Zouhairi	FastAPI, Docker, CI/CD, déploiement, monitoring

Table 1.1: Répartition des rôles du projet

1.3 Mise en place du projet dans Jira

Le projet Scrum est configuré dans Jira afin de centraliser les Epics, les User Stories, les tâches, les sprints et le suivi de l'avancement (espace *observatoire-prix-immobiliers*).

CHAPTER 2

Structure Agile du projet

2.1 Hiérarchie Jira

La hiérarchie retenue dans Jira est la suivante :



Les activités propres à la gestion Agile du Scrum Master sont quant à elles créées sous forme de **TASK – SCRUM MASTER** (label `scrum-master`), ce qui permet de ne pas mélanger les fonctionnalités du produit avec les activités de gestion du projet. Cette distinction rend visible et traçable, de façon indépendante, le travail de pilotage réalisé par le Scrum Master tout au long des quatre sprints.

2.2 Les quatre Epics

primaryblue!15 Epic	Nom	Description
EPIC-1	Data Acquisition & Data Strategy	Collecte, documentation, ingestion et stockage initial des données immobilières
EPIC-2	Data Transformation & Quality	Transformation, nettoyage, Data Quality, tests et orchestration
EPIC-3	Machine Learning & MLflow	Feature engineering, expérimentation, évaluation et reproductibilité des modèles
EPIC-4	Deployment & Observability	FastAPI, Docker, CI/CD, déploiement, monitoring et observabilité

Table 2.1: Epics du projet

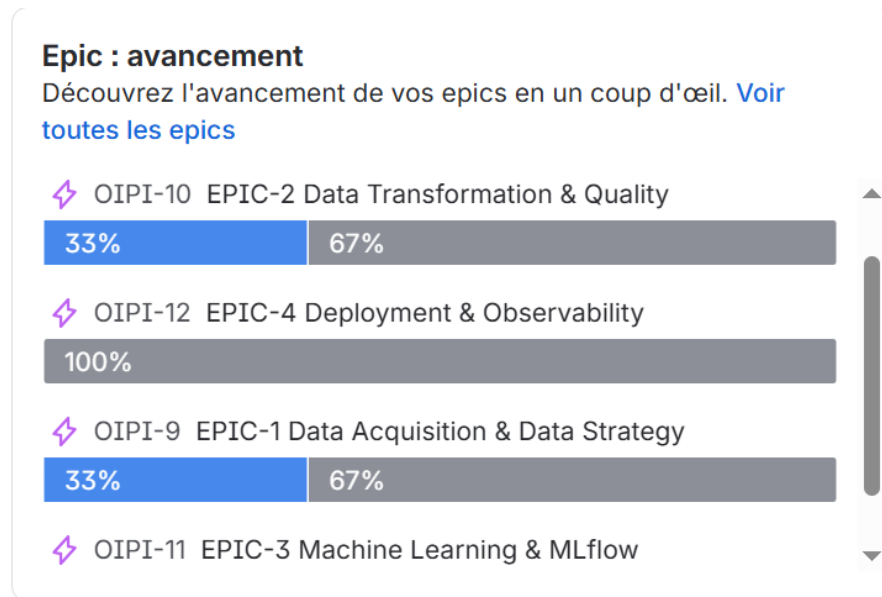


Figure 2.1: Les quatre Epics configurés dans Jira

2.3 Product Backlog

Le Product Backlog contient 24 User Stories réparties sur les quatre sprints, pour un total de 95 story points.

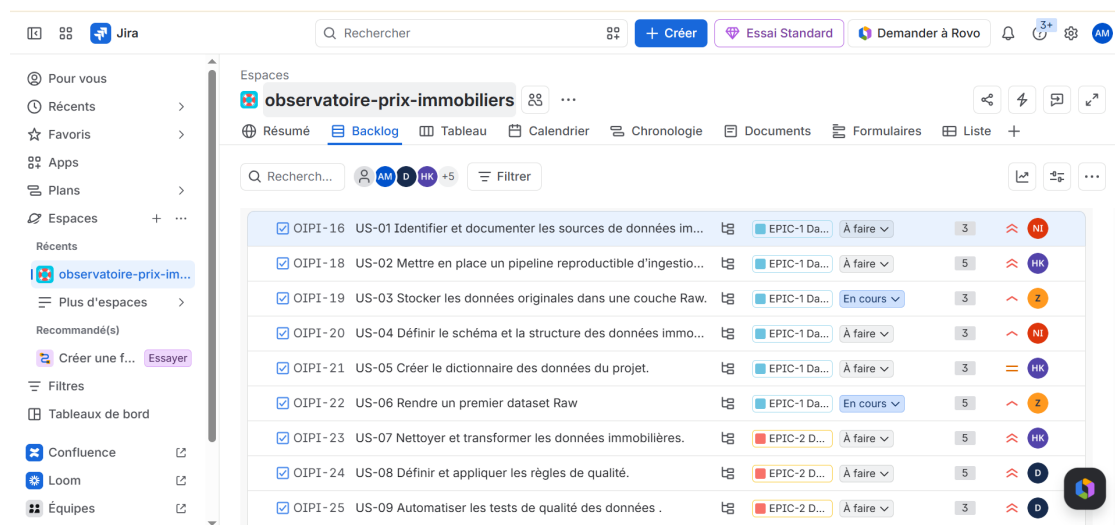


Figure 2.2: Vue générale du Product Backlog dans Jira

primaryblue!15 ID	Epic	Résumé	Priorité	SP	Sprint	Assigné
US-01	E1	Identifier et documenter les sources de données	Haute	3	S1	Nohaila Ichou
US-02	E1	Mettre en place le pipeline d'ingestion	Haute	5	S1	Hajar El Khalidi
US-03	E1	Stocker les données dans la couche Raw	Haute	3	S1	Zainab Rhouni

primaryblue!15 ID	Epic	Résumé	Priorité	SP	Sprint	Assigné
US-04	E1	Définir le schéma des données immobilières	Haute	3	S1	Hajar El Khalidi
US-05	E1	Créer le dictionnaire des données	Moyenne	3	S1	Nohaila Ichou
US-06	E1	Rendre un premier dataset Raw disponible	Haute	5	S1	Zainab Rhouni
US-07	E2	Nettoyer et transformer les données immobilières	Haute	5	S2	Zainab Rhouni
US-08	E2	Définir les règles de Data Quality	Haute	5	S2	Douaa Bounadar
US-09	E2	Automatiser les tests de qualité	Haute	3	S2	Douaa Bounadar
US-10	E2	Orchestrer le pipeline DataOps	Haute	5	S2	Hajar El Khalidi
US-11	E2	Produire un dataset Curated validé	Haute	3	S2	Zainab Rhouni
US-12	E2	Calculer les indicateurs immobiliers	Moyenne	3	S2	Zainab Rhouni
US-13	E3	Construire un modèle baseline	Haute	3	S3	Salama Boussaid
US-14	E3	Réaliser le feature engineering	Haute	5	S3	Salama Boussaid
US-15	E3	Tracer les expériences avec MLflow	Haute	5	S3	Malika Chaaban
US-16	E3	Comparer plusieurs modèles de régression	Haute	5	S3	Malika Chaaban
US-17	E3	Sélectionner et enregistrer le meilleur modèle	Haute	3	S3	Malika Chaaban
US-18	E3	Garantir la reproductibilité de l'entraînement	Haute	3	S3	Salama Boussaid
US-19	E4	Exposer le modèle avec FastAPI	Haute	5	S4	Youness Zouhairi
US-20	E4	Créer un endpoint Health Check	Moyenne	2	S4	Youness Zouhairi
US-21	E4	Conteneuriser l'application avec Docker	Haute	5	S4	Youness Zouhairi
US-22	E4	Mettre en place le CI/CD	Haute	5	S4	Youness Zouhairi
US-23	E4	Déployer le service de prédiction	Haute	3	S4	Youness Zouhairi
US-24	E4	Mettre en place le monitoring	Haute	5	S4	Youness Zouhairi

Table 2.2: Product Backlog complet (24 User Stories)

CHAPTER 3

Déroulement détaillé des sprints

Cette section reprend, pour chaque User Story, la User Story elle-même (le cas échéant), ses critères d'acceptation et le détail nominatif des quatre Subtasks Jira associées, tel que documenté dans le livrable. Cette granularité permet de tracer précisément la contribution de chaque membre.

3.1 Sprint 1 – Cadrage, Data Strategy et Ingestion

Définir le cadrage du projet, organiser l'équipe et mettre en place une première chaîne reproductible permettant de collecter et stocker les données immobilières dans une couche Raw.

Six User Stories composent ce sprint (US-01 à US-06), couvrant l'identification des sources de données, la mise en place du pipeline d'ingestion, le stockage dans la couche Raw, la définition du schéma, la création du dictionnaire de données et la mise à disposition du premier dataset Raw. Chaque User Story est décomposée en quatre Subtasks techniques assignées nominativement.

☐ Backlog (48 tickets)	79	16	0	Créer un sprint
☒ OIPI-16 US-01 Identifier et documenter les sources de données immobilières.	EPIC-1 Data Acqui...	À faire	3	NI
☒ OIPI-18 US-02 Mettre en place un pipeline reproductible d'ingestion des données.	EPIC-1 Data Acqui...	À faire	5	HK
☒ OIPI-19 US-03 Stocker les données originales dans une couche Raw.	EPIC-1 Data Acqui...	En cours	3	Z
☒ OIPI-20 US-04 Définir le schéma et la structure des données immobilières	EPIC-1 Data Acqui...	À faire	3	NI
☒ OIPI-21 US-05 Créer le dictionnaire des données du projet.	EPIC-1 Data Acqui...	À faire	3	HK
☒ OIPI-22 US-06 Rendre un premier dataset Raw	EPIC-1 Data Acqui...	En cours	5	Z
☒ OIPI-23 US-07 Nettoyer et transformer les données immobilières.	EPIC-2 Data Trans...	À faire	5	HK
☒ OIPI-24 US-08 Définir et appliquer les règles de qualité.	EPIC-2 Data Trans...	À faire	5	D
☒ OIPI-25 US-09 Automatiser les tests de qualité des données .	EPIC-2 Data Trans...	À faire	3	D
☒ OIPI-26 US-10 Orchestrer les étapes du pipeline DataOps.	EPIC-2 Data Trans...	En cours	5	Z
☒ OIPI-27 US-11 Produire un dataset Curated validé pour le Machine Learning.	EPIC-2 Data Trans...	À faire	3	HK
☒ OIPI-28 US-12 Produire les premiers indicateurs	EPIC-2 Data Trans...	En cours	3	Z
☒ OIPI-29 US-13 Construire un modèle baseline de prédiction des prix.	EPIC-3 Machine L...	À faire	3	SB
☒ OIPI-30 US-14 Réaliser le feature engineering.	EPIC-3 Machine L...	À faire	5	MC
☒ OIPI-31 US-15 Tracer les expériences avecMLFLOW	EPIC-3 Machine L...	À faire	5	MC
☒ OIPI-32 US-16 Comparer plusieurs modeles de régression	EPIC-3 Machine L...	À faire	5	SB
☒ OIPI-33 US-17 Sélectionner et enregistrer le meilleur modèle	EPIC-3 Machine L...	À faire	3	SB
☒ OIPI-34 US-18 Garantir la reproductibilité du pipeline d'entraînement.	EPIC-3 Machine L...	À faire	3	MC
☒ OIPI-35 US-19 Exposer le modèle à travers une API FastAPI.	EPIC-4 Deploye...	À faire	5	Y
☒ OIPI-36 US-20 Créer un endpoint permettant de contrôler l'état de l'API.	EPIC-4 Deploye...	À faire	2	Y
☒ OIPI-37 US-21 Conteneuriser l'application avec Docker	EPIC-4 Deploye...	À faire	5	Y
☒ OIPI-38 US-22 Mettre en place une chaîne CI/CD.	EPIC-4 Deploye...	À faire	5	Y
☒ OIPI-39 US-23 Déployer l'application dans un environnement accessible.	EPIC-4 Deploye...	À faire	3	Y
☒ OIPI-40 US-24 Mettre en place le monitoring et l'observabilité.	EPIC-4 Deploye...	À faire	5	Y
☒ OIPI-137 SM-01 Configurer le projet Scrum dans Jira		À faire	-	AM
☒ OIPI-138 SM-02 Créer et organiser les quatre Epics.		À faire	-	AM
☒ OIPI-139 SM-03 Créer et structurer le Product Backlog		À faire	-	AM
☒ OIPI-140 SM-04 Créer les quatre sprints dans Jira.		À faire	-	AM
☒ OIPI-141 SM-05 Préparer et animer le Sprint Planning 1		À faire	-	AM
☒ OIPI-142 SM-06 Suivre les obstacles et préparer la Review et la Retrospective 1.		À faire	-	AM
☒ OIPI-143 SM-07 Effectuer le Backlog Refinement avant le Sprint 2		À faire	-	AM
☒ OIPI-144 SM-08 Préparer et animer le Sprint Planning 2		À faire	-	AM
☒ OIPI-145 SM-09 Suivre les dépendances Data Engineering / Data Quality.		À faire	-	AM
☒ OIPI-146 SM-10 Maintenir le board Jira à jour.		À faire	-	AM
☒ OIPI-147 SM-11 Préparer et organiser la Sprint Review 2.		À faire	-	AM
☒ OIPI-148 SM-12 Animer et documenter la Retrospective 2.		À faire	-	AM
☒ OIPI-149 SM-13 Préparer le Sprint Planning 3.		À faire	-	AM
☒ OIPI-150 SM-14 Suivre la dépendance Dataset Curated vers Machine Learning		À faire	-	AM
☒ OIPI-151 SM-15 Coordonner le travail entre les deux ML Engineers.		À faire	-	AM
☒ OIPI-152 SM-16 Suivre les blocages relatifs à MLflow et aux modèles.		À faire	-	AM
☒ OIPI-153 SM-17 Organiser la Sprint Review 3		À faire	-	AM
☒ OIPI-154 SM-18 Animer et documenter la Retrospective 3		À faire	-	AM
☒ OIPI-155 SM-19 Préparer le Sprint Planning 4		À faire	-	AM
☒ OIPI-156 SM-20 Coordonner ML Engineers et DevOps Engineer		À faire	-	AM
☒ OIPI-157 SM-21 Suivre les blocages FastAPI, Docker et CI/CD		À faire	-	AM
☒ OIPI-158 SM-22 Vérifier la préparation du déploiement		À faire	-	AM
☒ OIPI-159 SM-23 Organiser la Sprint Review finale		À faire	-	AM
☒ OIPI-160 SM-24 Organiser la Retrospective finale et consolider les leçons apprises.		À faire	-	AM

+ Créer

Figure 3.1: Sprint Backlog du Sprint 1 dans Jira

US-01 – Identifier et documenter les sources

En tant que Product Owner, je souhaite identifier et documenter les sources de données immobilières afin de garantir leur pertinence pour les objectifs du projet.

Critères d'acceptation : au moins une source est retenue ; son origine est documentée ; son format est identifié ; les variables principales sont décrites ; les limitations sont documentées.

- ST-01.1 – Nohaila Ichou : rechercher les sources de données disponibles
- ST-01.2 – Hajar El Khalidi : analyser les variables disponibles
- ST-01.3 – Zainab Rhouni : analyser les formats et la structure des sources
- ST-01.4 – Nohaila Ichou : comparer les sources et documenter la source retenue

US-02 – Pipeline d'ingestion

En tant que Data Engineer, je souhaite automatiser l'ingestion des données afin de rendre leur collecte reproductible.

Critères d'acceptation : les données sont récupérées correctement ; l'exécution est reproductible ; les erreurs sont détectables ; les logs permettent de suivre l'exécution.

- ST-02.1 – Hajar El Khalidi : analyser le mécanisme d'accès aux données
- ST-02.2 – Hajar El Khalidi : développer le script d'ingestion
- ST-02.3 – Hajar El Khalidi : ajouter la gestion des erreurs et les logs
- ST-02.4 – Hajar El Khalidi : tester et documenter l'exécution du pipeline

US-03 – Couche Raw

En tant que Data Engineer, je souhaite conserver les données originales dans une couche Raw afin de garantir leur traçabilité.

- ST-03.1 – Zainab Rhouni : définir l'organisation de la couche Raw
- ST-03.2 – Zainab Rhouni : créer la structure de stockage
- ST-03.3 – Zainab Rhouni : sauvegarder les données originales
- ST-03.4 – Zainab Rhouni : tester la lecture et la récupération des données Raw

US-04 – Schéma des données

En tant que Data Engineer, je souhaite définir un schéma commun afin que l'ensemble de l'équipe puisse comprendre et exploiter correctement les données.

- ST-04.1 – Hajar El Khalidi : identifier les colonnes principales
- ST-04.2 – Zainab Rhouni : identifier les types de données
- ST-04.3 – Hajar El Khalidi : définir les champs obligatoires
- ST-04.4 – Zainab Rhouni : documenter le schéma final

US-05 – Dictionnaire de données

- ST-05.1 – Nohaila Ichou : lister les variables
- ST-05.2 – Nohaila Ichou : rédiger la description métier des variables
- ST-05.3 – Hajar El Khalidi : ajouter les types et formats
- ST-05.4 – Nohaila Ichou : consolider et publier le dictionnaire

US-06 – Premier dataset Raw disponible

- ST-06.1 – Hajar El Khalidi : exécuter le pipeline d'ingestion
- ST-06.2 – Zainab Rhouni : vérifier le stockage du dataset
- ST-06.3 – Hajar El Khalidi : vérifier la présence des colonnes attendues
- ST-06.4 – Zainab Rhouni : rendre le dataset accessible à l'équipe

3.2 Sprint 2 – Transformation, Qualité et Orchestration

Transformer les données Raw en un dataset fiable, validé, testé et prêt à être utilisé par les composants analytiques et Machine Learning.

Ce sprint (US-07 à US-12) constitue le cœur de la Partie II de ce rapport : c'est au cours de ce sprint que la Data Quality Engineer, Douaa Bounadar, définit les règles de Data Quality (US-08) et automatise les tests (US-09), pendant que Zainab Rhouni nettoie les données (US-07) et produit le dataset Curated (US-11), et que Hajar El Khalidi orchestre le pipeline (US-10). Le détail technique de ces travaux est présenté au Chapitre 5 (Partie II – Qualité des données & Data Contracts).

US-07 – Nettoyage et transformation

- ST-07.1 – Zainab Rhouni : standardiser les formats
- ST-07.2 – Zainab Rhouni : traiter les valeurs manquantes
- ST-07.3 – Hajar El Khalidi : identifier et traiter les doublons
- ST-07.4 – Zainab Rhouni : produire la couche Processed

US-08 – Règles de Data Quality

- ST-08.1 – Douaa Bounadar : définir les règles de complétude
- ST-08.2 – Douaa Bounadar : définir les règles de validité
- ST-08.3 – Douaa Bounadar : définir les règles d'unicité et de cohérence
- ST-08.4 – Douaa Bounadar : documenter les seuils de qualité

US-09 – Tests automatisés

- ST-09.1 – Douaa Bounadar : implémenter les tests de schéma
- ST-09.2 – Douaa Bounadar : implémenter les tests de valeurs
- ST-09.3 – Douaa Bounadar : implémenter les tests de doublons
- ST-09.4 – Douaa Bounadar : automatiser la génération du résultat des tests

US-10 – Orchestration

- ST-10.1 – Hajar El Khalidi : définir les différentes tâches du workflow
- ST-10.2 – Hajar El Khalidi : configurer les dépendances
- ST-10.3 – Hajar El Khalidi : gérer les erreurs d'exécution
- ST-10.4 – Hajar El Khalidi : tester l'orchestration de bout en bout

US-11 – Dataset Curated

- ST-11.1 – Zainab Rhouni : sélectionner les variables finales
- ST-11.2 – Douaa Bounadar : valider les contrôles de qualité
- ST-11.3 – Zainab Rhouni : produire le dataset Curated
- ST-11.4 – Douaa Bounadar : valider le dataset pour les ML Engineers

US-12 – Indicateurs immobiliers

- ST-12.1 – Zainab Rhouni : calculer le prix moyen
- ST-12.2 – Zainab Rhouni : calculer le prix médian
- ST-12.3 – Zainab Rhouni : calculer le prix moyen au mètre carré
- ST-12.4 – Zainab Rhouni : produire les agrégations géographiques disponibles

3.3 Sprint 3 – Machine Learning et MLflow

Construire, expérimenter et comparer plusieurs modèles de prédiction immobilière en garantissant la reproductibilité et la traçabilité des expériences avec MLflow.

Les User Stories US-13 à US-18 couvrent la construction d'un modèle baseline, le feature engineering, le tracking MLflow, la comparaison de plusieurs algorithmes de régression, la sélection du modèle final et la garantie de reproductibilité de l'entraînement. Trois modèles (Linear Regression, Random Forest, XGBoost) sont entraînés et comparés (cf. Partie III, Chapitres 12 et 13) ; le modèle **Random Forest Regressor** ($R^2 \approx 0.90$) est retenu par l'équipe et enregistré dans le Model Registry MLflow sous le nom `prix_immobilier_maroc`, avec l'alias `champion`.

US-13 – Modèle baseline

- ST-13.1 – Salama Boussaid : préparer le jeu d'entraînement et de test
- ST-13.2 – Salama Boussaid : sélectionner un algorithme baseline
- ST-13.3 – Salama Boussaid : entraîner le modèle
- ST-13.4 – Salama Boussaid : calculer MAE, RMSE et R^2

US-14 – Feature Engineering

- ST-14.1 – Salama Boussaid : analyser les variables explicatives
- ST-14.2 – Salama Boussaid : encoder les variables catégorielles
- ST-14.3 – Malika Chaaban : créer ou sélectionner les features pertinentes
- ST-14.4 – Salama Boussaid : intégrer les transformations dans un pipeline reproductible

US-15 – MLflow Tracking

- ST-15.1 – Malika Chaaban : configurer MLflow
- ST-15.2 – Malika Chaaban : enregistrer les paramètres
- ST-15.3 – Malika Chaaban : enregistrer les métriques
- ST-15.4 – Malika Chaaban : enregistrer les modèles et artefacts

US-16 – Comparaison des modèles

- ST-16.1 – Salama Boussaid : entraîner plusieurs algorithmes
- ST-16.2 – Malika Chaaban : enregistrer les runs dans MLflow
- ST-16.3 – Malika Chaaban : comparer les métriques
- ST-16.4 – Salama Boussaid & Malika Chaaban : documenter les résultats et le choix du modèle

US-17 – Modèle final

- ST-17.1 – Malika Chaaban : identifier le meilleur run
- ST-17.2 – Malika Chaaban : valider les performances
- ST-17.3 – Malika Chaaban : enregistrer le modèle sélectionné
- ST-17.4 – Malika Chaaban : tester le rechargement du modèle

US-18 – Reproductibilité

- ST-18.1 – Salama Boussaid : versionner le code d'entraînement
- ST-18.2 – Salama Boussaid : documenter les dépendances
- ST-18.3 – Salama Boussaid : centraliser la configuration
- ST-18.4 – Salama Boussaid : reproduire une expérience existante

3.4 Sprint 4 – FastAPI, Docker, CI/CD et Monitoring

Transformer le modèle sélectionné en un service de prédiction déployable, automatisé et observable.

Ce dernier sprint (US-19 à US-24), intégralement porté par Youness Zouhairi, couvre l'exposition du modèle via FastAPI, la création d'un endpoint Health Check, la conteneurisation Docker, la mise en place du CI/CD, le déploiement du service et l'ajout du monitoring. Le détail complet de ces travaux – qui vont bien au-delà du périmètre initial du sprint pour couvrir également l'orchestration Dagster et le déploiement Komodo – constitue la Partie III de ce rapport.

US-19 – FastAPI

- ST-19.1 – Youness Zouhairi : créer l'application FastAPI
- ST-19.2 – Malika Chaaban : préparer le chargement du modèle
- ST-19.3 – Youness Zouhairi : créer l'endpoint de prédiction
- ST-19.4 – Youness Zouhairi : valider et tester les requêtes

US-20 – Health Check

- ST-20.1 – Youness Zouhairi : créer l'endpoint `/health`
- ST-20.2 – Youness Zouhairi : définir la réponse de santé
- ST-20.3 – Youness Zouhairi : tester les codes HTTP
- ST-20.4 – Youness Zouhairi : préparer son utilisation par le monitoring

US-21 – Docker

- ST-21.1 – Youness Zouhairi : créer le Dockerfile
- ST-21.2 – Youness Zouhairi : configurer les dépendances
- ST-21.3 – Youness Zouhairi : construire et lancer l'image
- ST-21.4 – Youness Zouhairi : tester l'API dans le conteneur

US-22 – CI/CD

- ST-22.1 – Youness Zouhairi : configurer le déclenchement de la pipeline
- ST-22.2 – Youness Zouhairi : automatiser les tests
- ST-22.3 – Youness Zouhairi : automatiser le build Docker
- ST-22.4 – Youness Zouhairi : gérer les échecs de pipeline

US-23 – Déploiement

- ST-23.1 – Youness Zouhairi : préparer l'environnement cible
- ST-23.2 – Youness Zouhairi : déployer l'application
- ST-23.3 – Youness Zouhairi : vérifier l'accès à l'API
- ST-23.4 – Youness Zouhairi : effectuer un test de prédiction en production

US-24 – Monitoring et observabilité

- ST-24.1 – Youness Zouhairi : mettre en place les logs applicatifs
- ST-24.2 – Youness Zouhairi : surveiller la disponibilité et les erreurs
- ST-24.3 – Youness Zouhairi : surveiller le temps de réponse
- ST-24.4 – Malika Chaaban : définir les indicateurs liés aux données et prédictions

3.5 Tasks spécifiques au Scrum Master

En complément des User Stories fonctionnelles, 24 Tasks de type **Task - Scrum Master** rendent visibles et traçables les activités de planification, de coordination, de suivi des obstacles et d'animation des cérémonies Scrum, réparties sur les quatre sprints.

Sprint 1

primaryblue!15 ID	Task	Résultat attendu
SM-01	Configurer le projet Scrum dans Jira	Projet et board disponibles
SM-02	Créer et organiser les quatre Epics	Structure du backlog créée
SM-03	Créer et structurer le Product Backlog	24 User Stories enregistrées
SM-04	Créer les quatre sprints dans Jira	Planning initial disponible
SM-05	Préparer et animer le Sprint Planning 1	Sprint Goal et Sprint Backlog définis
SM-06	Suivre les obstacles et préparer la Review et la Retrospective 1	Bilan du Sprint 1 documenté

Table 3.1: Tasks Scrum Master – Sprint 1

Sprint 2

primaryblue!15 ID	Task	Résultat attendu
SM-07	Effectuer le Backlog Refinement avant le Sprint 2	Stories clarifiées
SM-08	Préparer et animer le Sprint Planning 2	Sprint 2 planifié
SM-09	Suivre les dépendances Data Engineering / Data Quality	Blocages identifiés tôt
SM-10	Maintenir le board Jira à jour	Avancement visible
SM-11	Préparer et organiser la Sprint Review 2	Incrément évalué
SM-12	Animer et documenter la Retrospective 2	Actions d'amélioration définies

Table 3.2: Tasks Scrum Master – Sprint 2

Sprint 3

primaryblue!15 ID	Task	Résultat attendu
SM-13	Préparer le Sprint Planning 3	Objectifs ML clairement définis
SM-14	Suivre la dépendance Dataset Curated vers Machine Learning	Dataset disponible à temps
SM-15	Coordonner le travail entre les deux ML Engineers	Expériences réparties
SM-16	Suivre les blocages relatifs à MLflow et aux modèles	Impediments traités

primaryblue!15 ID	Task	Résultat attendu
SM-17	Organiser la Sprint Review 3	Modèle candidat présenté
SM-18	Animer et documenter la Retrospective 3	Actions Sprint 4 identifiées

Table 3.3: Tasks Scrum Master – Sprint 3

Sprint 4

primaryblue!15 ID	Task	Résultat attendu
SM-19	Préparer le Sprint Planning 4	Sprint Backlog final défini
SM-20	Coordonner ML Engineers et DevOps Engineer	Modèle correctement intégré
SM-21	Suivre les blocages FastAPI, Docker et CI/CD	Problèmes techniques visibles
SM-22	Vérifier la préparation du déploiement	Release prête pour validation
SM-23	Organiser la Sprint Review finale	Solution complète démontrée
SM-24	Organiser la Retrospective finale et consolider les leçons apprises	Bilan final documenté

Table 3.4: Tasks Scrum Master – Sprint 4

3.6 Planning global des sprints

primaryblue!15 Sprint	Objectif	Story Points
Sprint 1	Cadrage, Data Strategy, organisation Agile et ingestion	22
Sprint 2	Transformation, qualité des données, tests et orchestration	24
Sprint 3	Machine Learning, expérimentation et MLflow	24
Sprint 4	FastAPI, Docker, CI/CD, déploiement et monitoring	25
gray!15 Total		95

Table 3.5: Planning global des sprints

CHAPTER 4

Cérémonies, qualité de processus et suivi

4.1 Workflow du Board Jira

Le workflow utilisé pour le suivi des tickets est :

TO DO → IN PROGRESS → REVIEW → DONE

Les tickets sont déplacés progressivement selon leur état réel. Le Scrum Master vérifie que le board reste à jour afin qu'il représente fidèlement l'avancement réel de l'équipe.

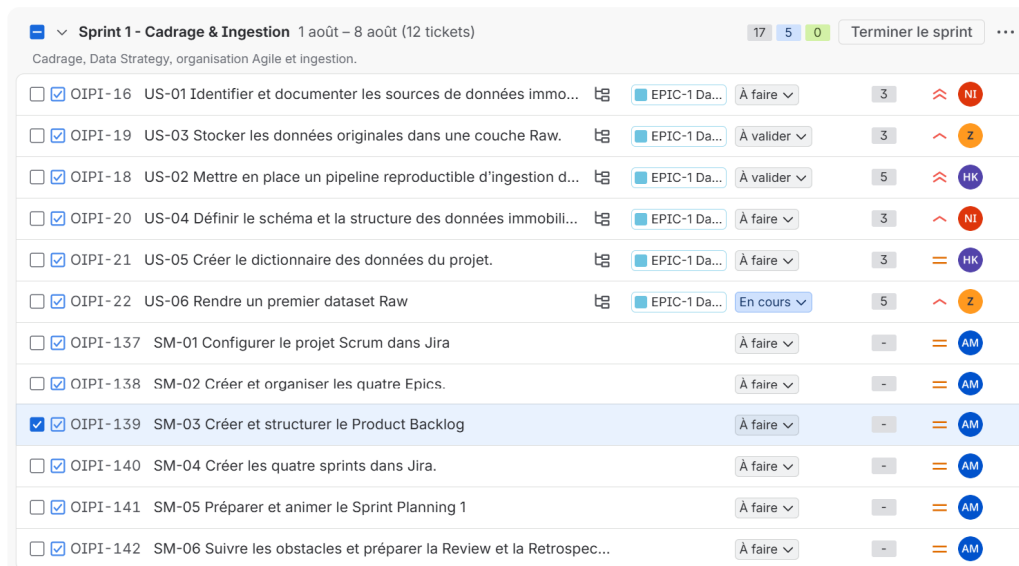


Figure 4.1: Board Jira pendant le Sprint 1

4.2 Cérémonies Scrum

Sprint Planning définir le Sprint Goal, sélectionner les User Stories, vérifier leur priorité et leurs critères d'acceptation, identifier les Subtasks, répartir les responsabilités et les dépendances.

Daily Scrum / Point de suivi chaque membre indique ce qu'il a terminé, ce qu'il prévoit de réaliser et les obstacles rencontrés (fréquence adaptée aux disponibilités de l'équipe dans le contexte académique).

Backlog Refinement clarification des stories, ajustement des priorités, révision des estimations, identification de nouvelles dépendances, découpage des éléments trop complexes.

Sprint Review présentation de l'incrément réalisé et vérification des critères d'acceptation.

Sprint Retrospective amélioration continue de l'organisation de l'équipe selon trois axes :
Went Well, To Improve, Actions.

4.3 Definition of Ready

Une User Story peut être intégrée dans un sprint lorsque :

- son objectif est clairement compris ;
- sa valeur est identifiable ;
- ses critères d'acceptation sont définis ;
- ses dépendances principales sont identifiées ;
- sa priorité est connue ;
- son estimation est disponible ;
- elle peut être décomposée en Subtasks réalisables.

4.4 Definition of Done

Une User Story peut être considérée comme terminée lorsque :

- toutes les Subtasks nécessaires sont terminées ;
- le développement est disponible dans le dépôt Git ;
- les tests nécessaires ont été exécutés ;
- les tests critiques sont validés ;
- les critères d'acceptation sont respectés ;
- la documentation nécessaire est présente ;
- aucune anomalie bloquante connue ne subsiste ;
- la fonctionnalité peut être démontrée pendant la Review.

4.5 Sprint Reviews

Sprint Review 1

primaryblue!15 Élément évalué	Statut	Commentaire
Vision du projet	Validé	La vision, la problématique, les objectifs, les utilisateurs cibles et la valeur métier ont été définis et documentés
Data Strategy	Validé	La stratégie de collecte, de stockage, de structuration et de qualité des données a été définie pour guider les étapes suivantes
Product Backlog	Validé	Le Product Backlog a été structuré dans Jira avec les Epics, User Stories, priorités, Story Points et responsables
Projet Jira	Validé	Le projet Scrum, le board, les quatre sprints et les principales règles de suivi ont été configurés
Pipeline d'ingestion	Validé	Le mécanisme d'ingestion a été mis en place et testé afin de récupérer les données de manière reproductible

primaryblue!15 Élément évalué	Statut	Commentaire
Données Raw	Validé	Un premier jeu de données brutes a été stocké dans la couche Raw et rendu accessible aux membres concernés

Table 4.1: Sprint Review 1

Sprint Review 2

primaryblue!15 Élément évalué	Statut	Commentaire
Transformation des données	Validé	Les données Raw ont été nettoyées, standardisées et transformées afin de produire une couche de données exploitable
Data Quality	Validé	Les règles de complétude, validité, unicité et cohérence ont été définies puis appliquées aux données
Tests automatisés	Validé	Les principaux contrôles de schéma, de valeurs et de doublons ont été automatisés afin de détecter rapidement les anomalies
Orchestration	Validé	Les étapes du pipeline ont été ordonnées et leurs dépendances configurées pour permettre une exécution cohérente de bout en bout
Dataset Curated	Validé	Un dataset Curated validé par les contrôles de qualité a été produit et mis à disposition de l'équipe Machine Learning

Table 4.2: Sprint Review 2

Sprint Review 3

primaryblue!15 Élément évalué	Statut	Commentaire
Modèle baseline	Validé	Un premier modèle de référence a été entraîné afin de disposer d'un niveau de performance initial pour les comparaisons
Feature engineering	Validé	Les variables numériques et catégorielles ont été préparées et les transformations utiles intégrées dans le pipeline ML
MLflow	Validé	Les expériences ont été tracées avec leurs paramètres, métriques, artefacts et modèles afin d'assurer leur traçabilité
Comparaison des modèles	Validé	Plusieurs modèles de régression ont été évalués avec des métriques communes afin de comparer leurs performances
Modèle final	Validé	Le modèle présentant le meilleur compromis a été sélectionné, enregistré et préparé pour l'intégration et le déploiement

primaryblue!15 Élément évalué	Statut	Commentaire
----------------------------------	--------	-------------

Table 4.3: Sprint Review 3

Sprint Review 4

primaryblue!15 Élément évalué	Statut	Commentaire
FastAPI	Validé	Le modèle a été exposé à travers une API avec un endpoint de prédiction et un endpoint de vérification de l'état du service
Docker	Validé	L'application a été conteneurisée afin de disposer d'un environnement d'exécution reproductible
CI/CD	Validé	Une chaîne d'intégration continue a été mise en place pour automatiser les tests et vérifications avant déploiement
Déploiement	Validé	Le service de prédiction a été rendu accessible dans l'environnement cible et testé avec des requêtes de validation
Monitoring	Validé	Des mécanismes de suivi de la disponibilité, des erreurs, du temps de réponse et des principales métriques applicatives ont été intégrés

Table 4.4: Sprint Review 4

4.6 Sprint Retrospectives

Sprint Retrospective 1

primaryblue!15 Went Well	To Improve	Actions Sprint 2
Bonne répartition des rôles, configuration de Jira réussie, Product Backlog structuré et première ingestion réalisée	Certaines tâches étaient encore trop larges et les mises à jour du board n'étaient pas toujours faites immédiatement	Découper davantage les tâches, mettre à jour Jira plus régulièrement et préparer les contrôles Data Quality dès le début du Sprint 2

Table 4.5: Sprint Retrospective 1

Sprint Retrospective 2

primaryblue!15 Went Well	To Improve	Actions Sprint 3
Bonne collaboration entre Data Engineers et Data Quality, tests automatisés opérationnels et dataset Curated produit	Quelques dépendances entre transformation et validation ont ralenti certaines activités ; la documentation devait être mieux centralisée	Clarifier les dépendances avant le Sprint Planning, figer le schéma du dataset Curated et préparer un passage propre vers l'équipe ML

Table 4.6: Sprint Retrospective 2

Sprint Retrospective 3

primaryblue!15 Went Well	To Improve	Actions Sprint 4
Les expériences ML ont été tracées avec MLflow, plusieurs modèles ont été comparés et un modèle candidat a été sélectionné	La gestion des noms d'expériences et la comparaison de certains runs pouvaient être plus homogènes et mieux planifiées	Standardiser les conventions MLflow, valider tôt le modèle à exposer et définir à l'avance le contrat d'entrée/sortie de l'API

Table 4.7: Sprint Retrospective 3

Sprint Retrospective 4

primaryblue!15 Went Well	To Improve	Leçons apprises
L'intégration FastAPI, Docker, CI/CD et monitoring a permis d'obtenir une chaîne MLOps complète et démontrable	Les dernières étapes d'intégration ont demandé davantage de tests entre le modèle, l'API et l'environnement de déploiement	Commencer les tests d'intégration plus tôt, automatiser au maximum les validations et prévoir le monitoring dès la conception

Table 4.8: Sprint Retrospective 4

4.7 Registre des obstacles

Le Scrum Master suit les obstacles pouvant ralentir l'équipe.

primaryblue!15 ID	Obstacle	Impact	Action	Statut
IMP-01	Configuration initiale du projet Jira et organisation du backlog	Moyen	Vérification de la structure du projet, création progressive des Epics, Stories, sprints et Tasks Scrum Master	Résolu

primaryblue!15 ID	Obstacle	Impact	Action	Statut
IMP-02	Hétérogénéité de certaines données et besoin de clarifier le schéma	Élevé	Définition d'un schéma commun, ajout de règles de qualité et coordination entre Data Engineering et Data Quality	Résolu
IMP-03	Dépendances d'intégration entre le modèle ML, FastAPI et l'environnement de déploiement	Élevé	Définition des interfaces, tests d'intégration et utilisation de Docker pour homogénéiser l'environnement d'exécution	Résolu

Table 4.9: Registre des impediments

4.8 Gestion des risques

primaryblue!15 ID	Risque	Prob.	Impact	Action préventive
R-01	Données de qualité insuffisante	Moyenne	Élevé	Contrôles Data Quality (cf. Partie II)
R-02	Retard dans l'ingestion ou la transformation	Moyenne	Élevé	Suivi des dépendances
R-03	Performances ML insuffisantes	Moyenne	Moyen	Comparer plusieurs modèles
R-04	Difficulté d'intégration ML / API	Moyenne	Élevé	Tester l'intégration progressivement
R-05	Problème de déploiement ou CI/CD	Moyenne	Élevé	Tester Docker et la pipeline avant fin de sprint

Table 4.10: Principaux risques identifiés et actions préventives

Ce registre de risques anticipe directement les difficultés rencontrées et documentées dans la Partie III (Chapitre 18 – Erreurs rencontrées et solutions appliquées), ce qui illustre la cohérence entre la planification Agile initiale et l'exécution technique finale du projet.

4.9 Suivi de l'avancement

primaryblue!15 Sprint	Stories prévues	Stories Done	Stories reportées	SP réalisés
Sprint 1	6	6	0	22 / 22
Sprint 2	6	6	0	24 / 24
Sprint 3	6	6	0	24 / 24
Sprint 4	6	6	0	25 / 25

Table 4.11: Suivi des quatre sprints

L'ensemble des 24 User Stories planifiées ont été terminées sans report, pour un total de 95 story points réalisés sur 95 planifiés, soit un taux de réalisation de 100 % sur les quatre sprints.

4.10 Conclusion de la partie Agile

La gestion Agile de l'Observatoire Intelligent des Prix Immobiliers repose sur une organisation hiérarchique claire (Epic → User Story → Subtasks). Les 24 User Stories représentent les principaux besoins fonctionnels et techniques du projet, et leur décomposition en 96 Subtasks nominatives a permis de transformer ces besoins en actions concrètes et assignables. En parallèle, les 24 Tasks spécifiques au Scrum Master rendent visibles les activités de planification, de coordination, de suivi des obstacles et d'animation des cérémonies Scrum. Les Sprint Reviews et Retrospectives, documentées à l'issue de chacun des quatre sprints, confirment l'atteinte systématique des Sprint Goals ainsi qu'une amélioration continue du fonctionnement de l'équipe. Jira constitue ainsi la source principale de suivi du travail Agile tout au long des quatre sprints, dont les livrables techniques sont détaillés dans les deux parties suivantes de ce rapport.

Part II

Qualité des Données & Data Contracts

CHAPTER 5

Stratégie de qualité des données

Rôle : Membre 5 – Data Quality Engineer (Douaa Bounadar), livré durant le Sprint 2 – Transformation, Qualité et Orchestration, en binôme avec le Data Engineer 2 (Zainab Rhouni) qui a construit la modélisation dbt sous-jacente.

5.1 Introduction et enjeux

Dans le cadre du pipeline DataOps / MLOps de l’Observatoire Immobilier, la Qualité des Données représente la passerelle critique entre la transformation (dbt) et la modélisation Machine Learning (scikit-learn / MLflow). Pour éviter le phénomène de Garbage In, Garbage Out, le binôme Data Engineer 2 / Data Quality Engineer a mis en place une stratégie de validation automatisée en deux temps – modélisation en couches d’abord, puis contrôles de qualité – afin de garantir que chaque donnée consommée par les ML Engineers soit certifiée, cohérente et entièrement traçable.

L’approche adoptée repose sur dbt (Data Build Tool) connecté à la base de données locale DuckDB (`morocco_housing_ingestion.duckdb`), la même base que celle utilisée en amont par l’ingestion (Partie I) et en aval par l’entraînement et le déploiement (Partie III).

CHAPTER 6

Modélisation dbt en couches : Staging, Intermediate, Marts

6.1 Contexte et objectif de la mission

En tant que Data Engineer 2, la mission a consisté à reprendre les données brutes ingérées par le Data Engineer 1 (via dlt dans DuckDB, cf. Partie I, US-02) et à les transformer en données exploitables pour le reste de l'équipe. Une modélisation complète a été mise en place avec dbt sur DuckDB, avec un triple objectif : nettoyer les données brutes (Staging), agréger les indicateurs clés (Intermediate) et produire des tables finales (Marts) répondant aux besoins du projet – prix au m², prix par ville et prix par type de bien – tout en garantissant la qualité des données via des tests automatisés et en documentant automatiquement le pipeline.

6.2 Architecture en couches (Medallion Architecture)

- **Couche Staging (nettoyage)** : la table `stg_housing_listings` filtre les annonces invalides (prix ≤ 0 ou surface ≤ 0), recalcule le prix au m² (`price_per_sqm_calculated`) pour garantir sa cohérence, et standardise les colonnes.
- **Couche Intermediate (agrégations)** : les tables `int_housing_by_city` et `int_housing_by_type` calculent les moyennes, minimums, maximums et nombres d'annonces respectivement par ville et par type de bien.
- **Couche Marts (produits finaux)** : trois tables finales sont exposées aux membres de l'équipe, notamment au ML Engineer 1 pour l'entraînement du modèle.

primaryblue!15 Table (Marts)	Contenu
<code>mart_price_per_sqm_by_city</code>	Classement des villes par prix au m ² (du plus élevé au plus bas)
<code>mart_price_by_city</code>	Prix moyen, minimum et maximum par ville
<code>mart_price_by_type</code>	Prix moyen par type de bien (Appartement, Villa, Bureau, etc.)

Table 6.1: Tables Marts produites par la couche de transformation dbt

6.3 Tests de qualité génériques (17 tests)

Pour garantir la fiabilité des données, 17 tests automatisés ont été implémentés directement dans dbt (fichier `schema.yml`), couvrant l'intégralité des modèles :

- **not_null** : vérifie que les colonnes critiques (identifiant, prix, surface, ville, type de bien) ne contiennent aucune valeur nulle.
- **unique** : garantit l'unicité des identifiants des annonces ainsi que des clés d'agrégation (villes et types de biens dans les tables intermédiaires et finales).
- **accepted_values** : contrôle que le champ **property_type** ne contient que des valeurs autorisées (Appartement, Villa, Maison, Bureau, etc.).

L'exécution de **dbt test** valide l'intégralité des 17 tests structurels avec un score parfait (17/17 PASS, 0 erreur). Ce résultat prouve la cohérence et la propreté des données avant leur utilisation par le modèle de Machine Learning. La Data Quality Engineer complète ensuite ce socle avec deux tests métier supplémentaires, portant le total à 19 tests – le chiffre confirmé de façon indépendante lors de l'exécution industrialisée du pipeline (Partie III).

6.4 Documentation, lignage et validation concrète

La documentation a été générée via **dbt docs generate**, permettant de consulter le dictionnaire des données ainsi que le graphe de lignage (*lineage*), qui illustre le chemin parcouru par les données depuis la table brute **housing_listings** jusqu'aux trois marts finaux. Ce lignage couvre la partie transformation (dbt) ; l'orchestration globale du projet (ingestion → transformation → ML) est assurée par Dagster (Partie III).

À titre de validation concrète, une requête directe sur la table **mart_price_per_sqm_by_city** confirme la disponibilité et la cohérence des données : Marrakech, Casablanca et Tanger ressortent comme les villes au prix au m² le plus élevé.

6.5 Conclusion

- **Code source** : l'intégralité des modèles SQL dbt versionnée sur GitHub, branche **feature/dbt-transformation**.
- **Qualité des données** : 17 tests dbt automatisés, tous validés.
- **Documentation** : documentation dbt générée, incluant descriptions et lignage des données.
- **Disponibilité** : les trois tables sont accessibles dans DuckDB et directement consommables par le ML Engineer 1.

CHAPTER 7

Data Contracts et tests de qualité

En complément des 17 tests structurels mis en place par le Data Engineer 2, la Data Quality Engineer valide le contrat de données (*Data Contract*) au moyen de deux tests de cohérence métier customisés, portant la couverture totale à 19 tests.

7.1 Tests génériques (structure & intégrité) – rappel

- **Unicité (unique)** : vérification de l'unicité de `listing_id`.
- **Complétude (not_null)** : absence de valeurs nulles sur les features essentielles (`price_mad`, `surface_m2`, `property_type`, `city`).
- **Valeurs acceptées (accepted_values)** : conformité du type de bien (Appartement, Villa, Maison, Terrain, etc.).

7.2 Tests customisés SQL (validité & cohérence métier)

Deux tests singuliers en SQL ont été développés dans le répertoire `transform/tests/`.

Cohérence des surfaces – `assert_surface_logique.sql`

Ce test vérifie qu'aucun bien n'affiche une surface négative ou nulle, et impose une surface minimale de 9 m² pour les appartements :

```
SELECT
    listing_id,
    property_type,
    surface_m2
FROM {{ ref('stg_housing_listings') }}
WHERE surface_m2 <= 0
    OR (property_type = 'Appartement' AND surface_m2 < 9)
```

SQL

```
SELECT
    listing_id,
    price_mad,
    surface_m2,
    price_per_sqm_calculated
FROM {{ ref('stg_housing_listings') }}
WHERE surface_m2 > 0
    AND ABS((price_mad / surface_m2) - price_per_sqm_calculated) > 2
```

Figure 7.1: Test SQL assert_calcul_prix_m2.sql

Validation du calcul du prix au m² – assert_calcul_prix_m2.sql

Ce test garantit la validité mathématique entre le prix total, la surface et le prix au m² calculé, avec une tolérance sur les arrondis :

```
SELECT
    listing_id,
    price_mad,
    surface_m2,
    price_per_sqm_calculated
FROM {{ ref('stg_housing_listings') }}
WHERE surface_m2 > 0
    AND ABS((price_mad / surface_m2) - price_per_sqm_calculated) > 2
```

SQL

```
SELECT
    listing_id,
    property_type,
    surface_m2
FROM {{ ref('stg_housing_listings') }}
WHERE surface_m2 <= 0
    OR (property_type = 'Appartement' AND surface_m2 < 9)
```

Figure 7.2: Test SQL assert_surface_logique.sql

19/19 tests validés avec succès (0 erreur). Ce résultat est confirmé de façon indépendante dans le rapport final d'industrialisation (Partie III), où l'exécution de `dbt run` en environnement conteneurisé aboutit au même total de 19 tests sur 6 modèles.

CHAPTER 8

Documentation, dictionnaire de données et lineage

8.1 Documentation et dictionnaire de données

La documentation a été générée via `dbt docs generate` afin d'exposer un dictionnaire de données complet à l'ensemble de l'équipe, définissant clairement le rôle de chaque champ du modèle certifié `stg_housing_listings`.

primaryblue!15 Colonne	Type	Description / Règle de contrat
<code>listing_id</code>	VARCHAR	Identifiant unique global de l'annonce (Unique, Not Null)
<code>price_mad</code>	DOUBLE	Prix total du bien en Dirhams Marocains (> 0)
<code>surface_m2</code>	DOUBLE	Surface habitable du bien en m^2 (> 0)
<code>price_per_sqm_calculated</code>	DOUBLE	Prix au m^2 calculé et validé mathématiquement
<code>property_type</code>	VARCHAR	Type de bien immobilier (Appartement, Villa, etc.)
<code>city</code>	VARCHAR	Ville de localisation de l'annonce
<code>loaded_at</code>	TIMESTAMP	Horodatage permettant le suivi de la fraîcheur

Table 8.1: Dictionnaire de données – Descriptions et contraintes du modèle Staging

Search for models...

Overview

Project Database Group

Sources

- raw_immobilier

Projects

- morocco_housing
 - models
 - intermediate
 - int_housing_by_city
 - int_housing_by_type
 - marts
 - mart_price_by_city
 - mart_price_by_type
 - mart_price_per_sqm_by_city
 - staging
 - stg_housing_listings
 - tests

stg_housing_listings view

Details Description Columns Referenced By Depends On Code

Column	Type	Description
rooms	BIGINT	
bedrooms	BIGINT	
bathrooms	BIGINT	
floor	BIGINT	
address	VARCHAR	
localisation	VARCHAR	
elevator	VARCHAR	
terrace	VARCHAR	
parking	VARCHAR	
property_type	VARCHAR	Type de bien
city	VARCHAR	Ville du bien
other_tags	VARCHAR	
loaded_at	TIMESTAMP WITH TIME ZONE	

Referenced By

Models Data Tests

Figure 8.1: Dictionnaire de données dbt : colonnes et types

8.2 Data Lineage (graphe de dépendances)

Le Data Lineage permet de visualiser l'intégralité du cycle de vie de la donnée au sein du projet.

Source (`raw_immobilier.housing_listings`) : données brutes issues de l'ingestion dlt (Data Engineer 1 – Partie I, US-02).

Staging (`stg_housing_listings`) : première couche de nettoyage produite par le Data Engineer 2. C'est sur ce modèle que s'appliquent les tests de qualité `assert_surface_logique` et `assert_calcul_prix_m2`.

Intermediate & Marts : les données validées alimentent ensuite les agrégations (`int_housing_by_city`, `int_housing_by_type`) et les tables finales consommateurs (`mart_price_by_city`, `mart_price_per_sqm_by_city`, `mart_price_by_type`) destinées au Machine Learning (Partie III).



Figure 8.2: Graphe de Data Lineage (DAG dbt)

CHAPTER 9

Conclusion et passation vers le MLOps

La mise en place de cette suite de tests dbt (17 tests structurels du Data Engineer 2 + 2 tests métier de la Data Quality Engineer = 19 tests), couplée à un Data Contract strict et à une documentation complète, permet d'élever le niveau de maturité DataOps du projet. Grâce à ces validations automatisées :

- 100 % des anomalies majeures (surfaces incohérentes, erreurs de calcul du prix au m²) ont été détectées et écartées avant toute modélisation ;
- les ML Engineers (Salama Boussaid et Malika Chaaban) disposent d'un dataset `stg_housing_listings` et de trois tables Marts certifiées, documentées et prêtes pour l'ingénierie des variables et l'entraînement des modèles ;
- le pipeline global est désormais totalement transparent et traçable via le Data Lineage, garantissant une maintenabilité optimale pour l'orchestration finale sous Dagster, décrite dans la Partie III.

Cette passation constitue le point d'articulation entre le travail de qualité des données réalisé au Sprint 2 et la modélisation Machine Learning, elle-même suivie de l'industrialisation complète de la chaîne présentée dans la suite de la Partie III.

Part III

Industrialisation, Déploiement et Monitoring

CHAPTER 10

Résumé exécutif et périmètre

Travail réalisé par Youness Zouhairi – Reconstruction locale, intégration DevOps, orchestration Dagster, FastAPI, Docker, CI/CD GitHub Actions, MLflow, déploiement Komodo et monitoring persistant. Branche de travail : `feature/devops-deployment`.

10.1 Résumé exécutif

Cette dernière partie présente de manière chronologique et technique l'ensemble du travail réalisé pour transformer le projet en une solution DataOps / MLOps cohérente, reproductible et déployable. L'objectif n'était pas uniquement de faire fonctionner un modèle de prédiction, mais de relier proprement toutes les briques produites par l'équipe durant les quatre sprints.

La solution finale s'appuie sur les composants suivants : dlt pour l'ingestion, DuckDB pour le stockage, dbt pour les transformations et les tests, Dagster pour l'orchestration, scikit-learn (Random Forest Regressor) pour la modélisation, MLflow pour le tracking et le Model Registry, FastAPI pour l'inférence, Docker Compose pour l'exécution multi-services, GitHub Actions pour la CI, Komodo pour le déploiement, puis un service dédié de monitoring pour la disponibilité, la latence, les métriques ML et la détection de dérive.

Validation finale confirmée :

- reconstruction locale fonctionnelle de la stack ;
- ingestion dlt opérationnelle vers DuckDB persistant ;
- 6/6 modèles dbt construits et 19/19 tests validés (cf. Partie II) ;
- modèle Random Forest Regressor (scikit-learn) entraîné, comparé et tracké dans MLflow ;
- pipeline Dagster complet exécuté avec succès sur le serveur ;
- modèle enregistré dans MLflow sous `prix_immobilier_maroc` avec alias `champion` ;
- API FastAPI saine, capable de recharger le modèle après redémarrage ;
- endpoint `POST /predict` retournant HTTP 200 avec un prix prédit cohérent ;
- monitoring persistant opérationnel (disponibilité, latence, dérive) ;
- CI GitHub Actions verte (lint, tests, validation dbt, build Docker).

10.2 Objectif du travail et périmètre de contribution

Le cahier des charges du module impose l'industrialisation du cycle de vie complet d'un projet Data / IA. Le travail présenté dans cette partie a consisté à :

1. Reconstruire localement le projet à partir des briques fournies par les autres membres de l'équipe ;
2. Unifier les chemins, variables d'environnement et dépendances pour que la chaîne fonctionnelle localement et dans Docker ;
3. Intégrer l'orchestration Dagster de bout en bout ;

4. Fiabiliser l'entraînement et l'enregistrement dans MLflow ;
5. Exposer proprement le modèle via FastAPI ;
6. Mettre en place la CI avec GitHub Actions ;
7. Déployer la stack sur Komodo ;
8. Compléter le projet avec un monitoring léger mais persistant ;
9. Résoudre les erreurs rencontrées jusqu'à obtention d'une exécution finale stable.

CHAPTER 11

Architecture globale de la solution finale

11.1 Chaîne fonctionnelle DataOps / MLOps

CSV / datasets → dlt (Ingestion) → DuckDB (Stockage) → dbt (Transformations) → dbt tests (Qualité) → ML (Entraînement) → MLflow (Registry) → FastAPI (Inférence)

Le bloc ML Entraînement s'appuie sur scikit-learn et retient un modèle Random Forest Regressor comme champion, après comparaison avec une régression linéaire et un modèle XGBoost, et tracking systématique des trois expérimentations dans MLflow.

11.2 Services du déploiement final

primaryblue!15 Service	Port interne	Port hôte	Rôle
api	8000	3900	FastAPI : santé, informations modèle, prédiction
mlflow	5000	3901	Tracking, artefacts, Model Registry, alias champion
dagster	3000	3902	Interface Dagster et définition du pipeline global
dagster-daemon	—	—	Exécution des runs Dagster en arrière-plan
monitoring	8001	3903	Disponibilité, latence, métriques ML et dérive PSI

Table 11.1: Services de la stack finale

La persistance repose sur plusieurs volumes : `housing_data`, `mlflow_data`, `mlflow_artifacts`, `dagster_data` et `monitoring_data`. Cette décision évite la perte de la base DuckDB, du suivi MLflow, des métadonnées Dagster et de la baseline / des journaux de monitoring.

CHAPTER 12

Modélisation Machine Learning : préparation des données et comparaison des modèles

Rôle : Membre 6 – ML Engineer 1 (Salama Boussaid), livré durant le Sprint 3.

12.1 Préparation des données

Les données utilisées proviennent de la table `stg_housing_listings` (1197 annonces immobilières, 19 colonnes). Le traitement appliqué avant modélisation a consisté en : suppression des lignes sans prix renseigné et des doublons ; filtrage des valeurs de prix aberrantes ($< 50\,000$ MAD ou $> 30\,000\,000$ MAD) ; exclusion des colonnes `address`, `elevator`, `terrace` et `parking` (98 % de valeurs manquantes) ; exclusion de `price_per_sqm_calculated` pour éviter la fuite de données ; imputation par la médiane des 25 valeurs manquantes de `rooms` ; regroupement des modalités rares de `localisation` (109 quartiers initialement) en une catégorie Autre au-delà d'un seuil de 5 occurrences, ramenant la variable à 49 catégories.

Les variables finalement retenues sont : `surface_m2`, `bedrooms`, `bathrooms`, `floor`, `rooms` (numériques), ainsi que `city`, `property_type` et `localisation_grouped` (catégorielles, One-Hot Encoding). La variable cible est `price_mad`.

12.2 Modèles entraînés

Trois modèles ont été entraînés et comparés sur un découpage 80 % train / 20 % test (`random_state=42`) : Linear Regression (baseline), Random Forest Regressor (200 arbres), XGBoost Regressor (300 estimateurs, learning rate 0.05).

12.3 Évaluation et comparaison des modèles

primaryblue!15 Modèle	RMSE (MAD)	MAE (MAD)	R^2
XGBoost	1 539 903	825 580	0.900
Linear Regression	1 645 188	902 793	0.885
Random Forest	1 734 450	912 237	0.873

Table 12.1: Comparaison des trois modèles (notebook d'expérimentation initial)

12.4 Modèle retenu

Décision finale de l'équipe : le modèle Random Forest Regressor a été retenu comme modèle de production, choix confirmé lors du tracking systématique des trois modèles dans MLflow,

où Random Forest a démontré la meilleure robustesse d'ensemble face aux valeurs extrêmes du marché immobilier marocain. Il s'agit du modèle effectivement chargé et servi en production (`model_type: RandomForestRegressor`).

CHAPTER 13

Tracking des expérimentations et Model Registry avec MLflow

Rôle : Membre 7 – ML Engineer 2 (Malika Chaaban), livré durant le Sprint 3.

13.1 Mise en place du tracking MLflow

Le backend de tracking retenu est SQLite (`sqlite:///mlflow.db`). Un run MLflow distinct a été créé pour chacun des trois modèles entraînés, avec enregistrement des paramètres, des métriques (RMSE, MAE, R^2) et du pipeline scikit-learn complet comme artefact.

primaryblue!15 Run MLflow	RMSE (MAD)	MAE (MAD)	R^2
linear_regression	1 571 716.56	849 020.50	0.903
random_forest	1 612 481.77	874 210.21	0.898
xgboost	1 932 469.47	962 490.53	0.853

Table 13.1: Métriques trackées dans MLflow pour les trois runs

13.2 Model Registry

Le modèle Random Forest Regressor a été enregistré dans le Model Registry MLflow sous le nom `prix_immobilier_maroc`, avec 3 versions enregistrées et l’alias `champion` assigné à la version Random Forest, effectivement chargée par l’API de production.

13.3 Détection de dérive (monitoring)

Une baseline statistique des features d’entraînement a été sauvegardée (`feature_baseline.json`), comparée à un nouveau lot de données via le PSI (*Population Stability Index*) :

- $\text{PSI} < 0.10$: pas de dérive significative (OK) ;
- $0.10 \leq \text{PSI} < 0.25$: dérive modérée, à surveiller ;
- $\text{PSI} \geq 0.25$: dérive importante, réentraînement à envisager.

Le script `ml/monitoring/drift_check.py` calcule ce score pour chaque feature et constitue la base méthodologique du service de monitoring persistant intégré en production (Chapitre 17), qui y ajoute un garde-fou `DRIFT_MIN_SAMPLES=20`.

CHAPTER 14

Reconstruction locale : démarche chronologique

14.1 Reprise du dépôt et compréhension de l'état initial

La première étape a consisté à reprendre le dépôt, comprendre les branches existantes et identifier les dépendances entre travaux de différents membres.

```
(.venv) PS C:\Users\sora7\OneDrive\Desktop\Web projects\mlops_meia_v2> git log --oneline --left-right --graph origin/main...HEAD
> ea2ee5c (HEAD -> feature/devops-deployment, origin/feature/devops-deployment) CHORE: ignore generated ML runtime artifacts
> 2a10878 Merge ML pipeline and MLflow for DevOps integration
> 6a54f0c (origin/feature/mlflow-tracking) FEAT: MLflow tracking, model registry, drift monitoring
< 5922fa7 (origin/main, origin/HEAD) Merge pull request #3 from hajar-elkhalidi/feature/ml-modelisation
| \
| < fd55597 (origin/feature/ml-modelisation) Delete docs/Rapport_Partie_Membre6_ML_Engineer1.docx
< d2b099e Merge pull request #2 from hajar-elkhalidi/feature/data-quality
● (.venv) PS C:\Users\sora7\OneDrive\Desktop\Web projects\mlops_meia_v2> git diff --name-status HEAD..origin/main
M      .gitignore
D      docs/Rapport_Partie_Membre6_ML_Engineer1.docx
D      ml/monitoring/drift_check.py
D      ml/tracking/comparaison_modeles.csv
D      ml/tracking/feature_baseline.json
D      ml/tracking/train_with_mlflow.py
M      requirements.txt
```

Figure 14.1: Analyse des écarts entre branche locale et origin/main

14.2 Reconstruction locale de la stack Docker

La stack a été relancée localement avec Docker Compose (`docker compose up -d`) pour vérifier que les services minimaux (`mlflow`, `api`, `dagster`) pouvaient redémarrer dans un environnement reproductible.

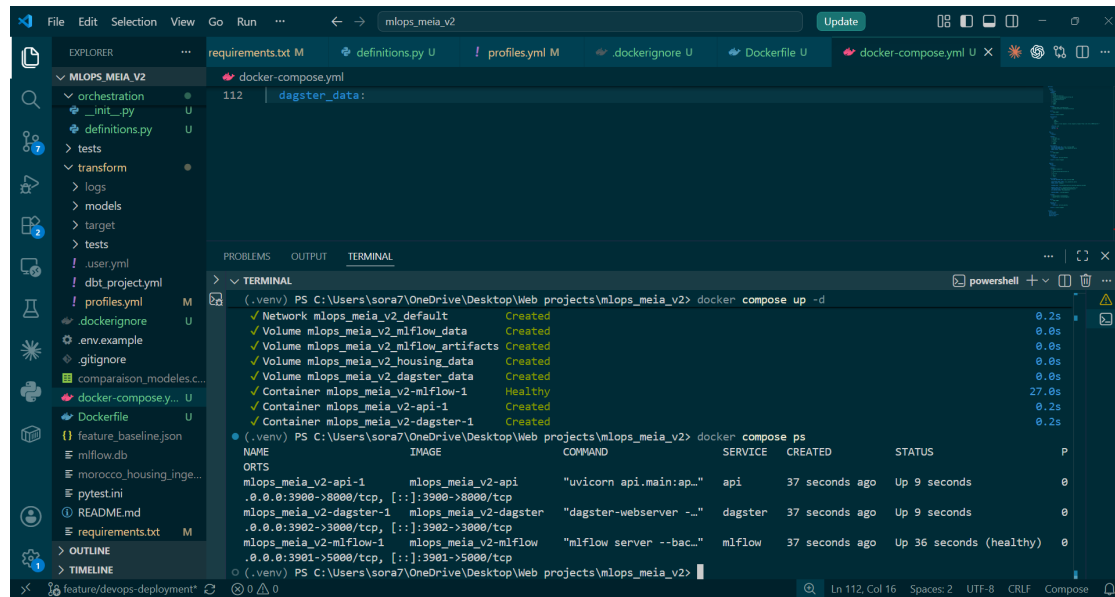


Figure 14.2: Exécution de docker compose up -d

14.3 Vérification de l'ingestion et de la base DuckDB

Les vérifications ont montré la présence de la table `raw_immobilier.housing_listings`, des tables internes `dlt` et d'un volume d'environ 1197 annonces.

```
(.venv) PS C:\Users\sora7\OneDrive\Desktop\Web projects\mlops_meia_v2\transform> dbt run --profiles-dir .
13:00:47 Running with dbt=1.12.2
13:00:48 Registered adapter: duckdb=1.11.0
13:00:48 Unable to do partial parsing because saved manifest not found. Starting full parse.
13:00:50 [WARNING][MissingArgumentsPropertyInGenericTestDeprecation]: Deprecated
functionality
Found top-level arguments to test `accepted_values` defined on
'stg_housing_listings' in package 'morocco_housing' (models/schema.yml).
Arguments to generic tests should be nested under the `arguments` property.
13:00:51 Found 6 models, 19 data tests, 1 source, 500 macros
13:00:51
13:00:51 Concurrency: 1 threads (target='dev')
13:00:51
13:00:51 1 of 6 START sql view model main.stg_housing_listings ..... [RUN]
13:00:51 1 of 6 OK created sql view model main.stg_housing_listings ..... [OK in 0.10s]
13:00:51 2 of 6 START sql table model main.int_housing_by_city ..... [RUN]
13:00:51 2 of 6 OK created sql table model main.int_housing_by_city ..... [OK in 0.07s]
13:00:51 3 of 6 START sql table model main.int_housing_by_type ..... [RUN]
13:00:52 3 of 6 OK created sql table model main.int_housing_by_type ..... [OK in 0.04s]
13:00:52 4 of 6 START sql table model main.mart_price_by_city ..... [RUN]
13:00:52 4 of 6 OK created sql table model main.mart_price_by_city ..... [OK in 0.05s]
13:00:52 5 of 6 START sql table model main.mart_price_per_sqm_by_city ..... [RUN]
13:00:52 5 of 6 OK created sql table model main.mart_price_per_sqm_by_city ..... [OK in 0.02s]
13:00:52 6 of 6 START sql table model main.mart price by type ..... [RUN]
```

Figure 14.3: Contrôle de la base DuckDB (a)

```
(.venv) PS C:\Users\sora7\OneDrive\Desktop\Web projects\mlops_meia_v2> Get-ChildItem api
-a----          16/08/2026    16:10          5006 model_service.py
-a----          16/08/2026    16:09          968 schemas.py
-a----          16/08/2026    15:59           0 __init__.py

(.venv) PS C:\Users\sora7\OneDrive\Desktop\Web projects\mlops_meia_v2> python -c "import api; print(api.__file__)"
C:\Users\sora7\OneDrive\Desktop\Web projects\mlops_meia_v2\api\__init__.py
(.venv) PS C:\Users\sora7\OneDrive\Desktop\Web projects\mlops_meia_v2> python -m pytest -q tests/test_api.py
... [100%]
===== warnings summary =====
.venv\Lib\site-packages\fastapi\testclient.py:1
C:\Users\sora7\OneDrive\Desktop\Web projects\mlops_meia_v2\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecatio
nWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
3 passed, 1 warning in 2.85s
(.venv) PS C:\Users\sora7\OneDrive\Desktop\Web projects\mlops_meia_v2>
```

Figure 14.4: Contrôle de la base DuckDB (b)

14.4 Validation des transformations dbt

Les tests dbt ont été intégrés dans la chaîne globale, avec un résultat final de **19 tests validés**, jouant le rôle de *quality gate* : si la qualité n'est pas satisfaisante, l'entraînement ne doit pas démarrer.

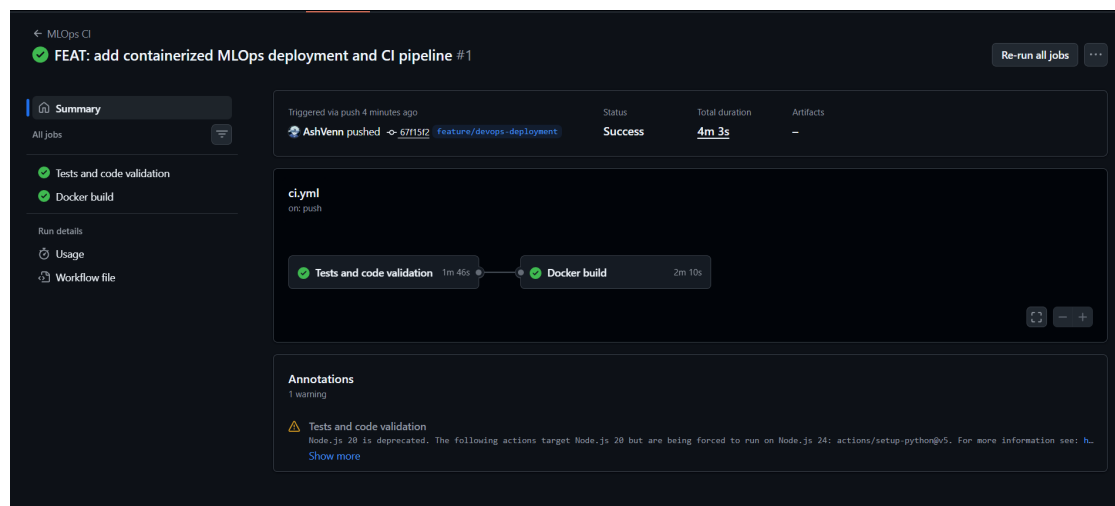


Figure 14.5: Exécution de dbt run

14.5 Validation de l'API et des tests automatiques

Les endpoints principaux sont GET /health, POST /predict, GET /model-info et GET /supported-values. L'API reste disponible même sans modèle champion : elle indique alors un état `waiting_for_model`.

14.6 Mise en place et validation de la CI

La CI exécute l'installation des dépendances, Ruff, pytest, dbt parse et le build Docker.

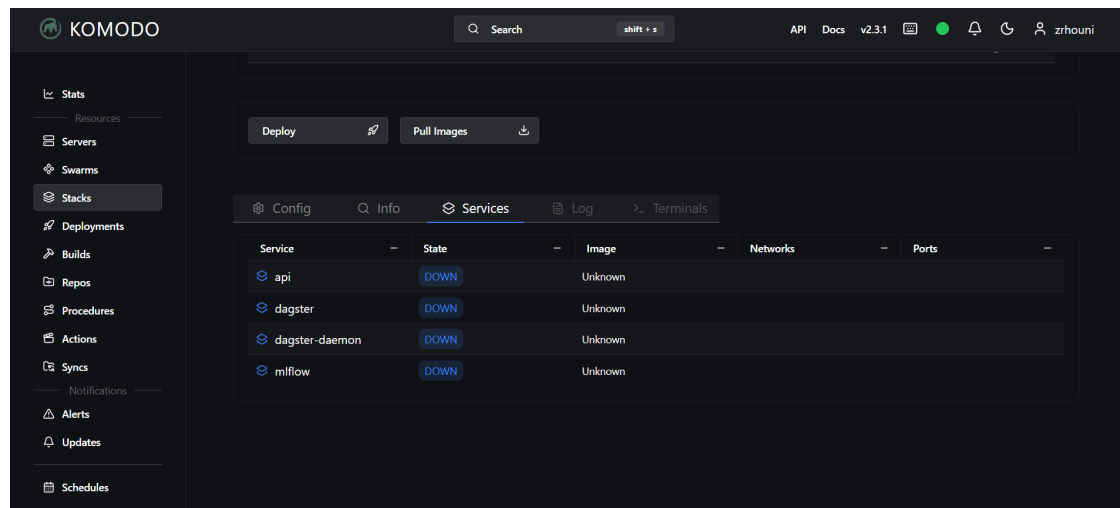


Figure 14.6: Pipeline CI GitHub Actions

CHAPTER 15

Industrialisation technique : unification de la chaîne

15.1 Correction des chemins de données et des variables d'environnement

La configuration a été pilotée par l'environnement (DUCKDB_PATH, SOURCE_TABLE, MLFLOW_EXPERIMENT, REGISTERED_MODEL_NAME, MODEL_ALIAS, MLFLOW_TRACKING_URI), supprimant les écarts entre comportement local et conteneurisé.

15.2 Intégration MLflow et gestion du modèle

L'alias `champion` pointe vers la version Random Forest Regressor du modèle `prix_immobilier_maroc`, permettant à l'API de charger `prix_immobilier_maroc@champion` sans modification applicative après un nouvel entraînement.

15.3 Orchestration Dagster de bout en bout

```
@dg.job
def full_mlops_pipeline():
    raw_data = ingestion()
    transformed_data = dbt_transform(raw_data)
    validated_data = data_quality(transformed_data)
    registered_model = train_and_register_model(validated_data)
    monitoring_ready(registered_model)
```

CHAPTER 16

Déploiement sur Komodo

16.1 Compréhension du comportement de la stack dans Komodo

La stack a été configurée à partir du dépôt Git avec la branche `feature/devops-deployment` et la plage de ports 39XX. Un *Update Stack* peut modifier la configuration sans démarrer les services, ce qui explique l'état `DOWN` observé initialement.

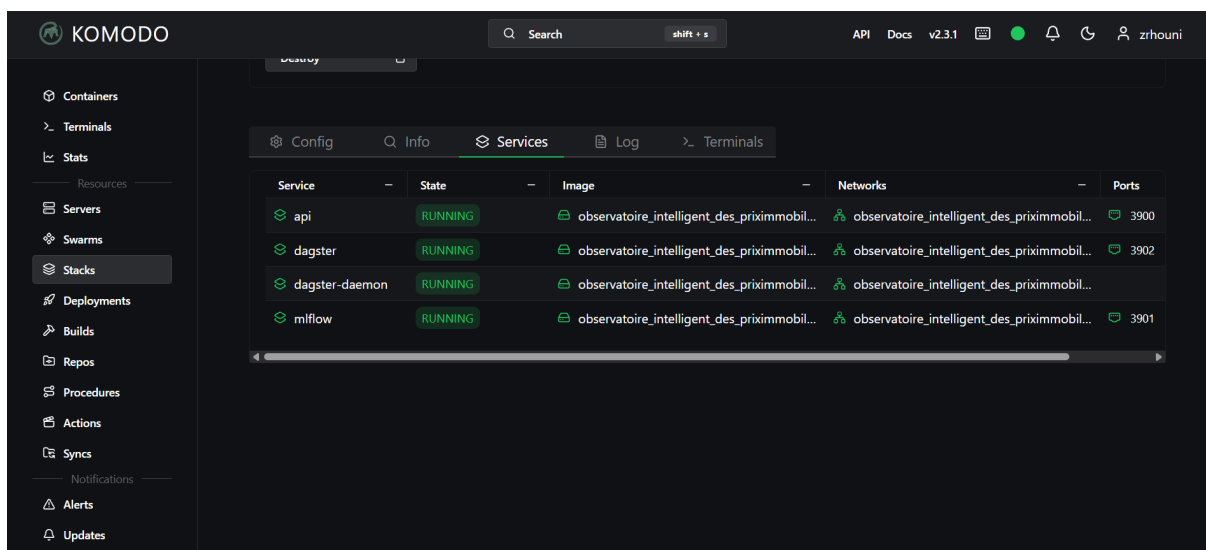


Figure 16.1: Services Komodo à l'état `DOWN`

16.2 Validation inter-services dans le conteneur Dagster

La communication réseau entre services a été validée depuis le shell Dagster : accès à `mlflow:5000`, `dagster:3000` et `api:8000/health`.

16.3 Exécution complète du pipeline sur le serveur

La séquence finale a confirmé l'enchaînement ingestion, dbt, qualité, entraînement, enregistrement MLflow et préparation du monitoring.

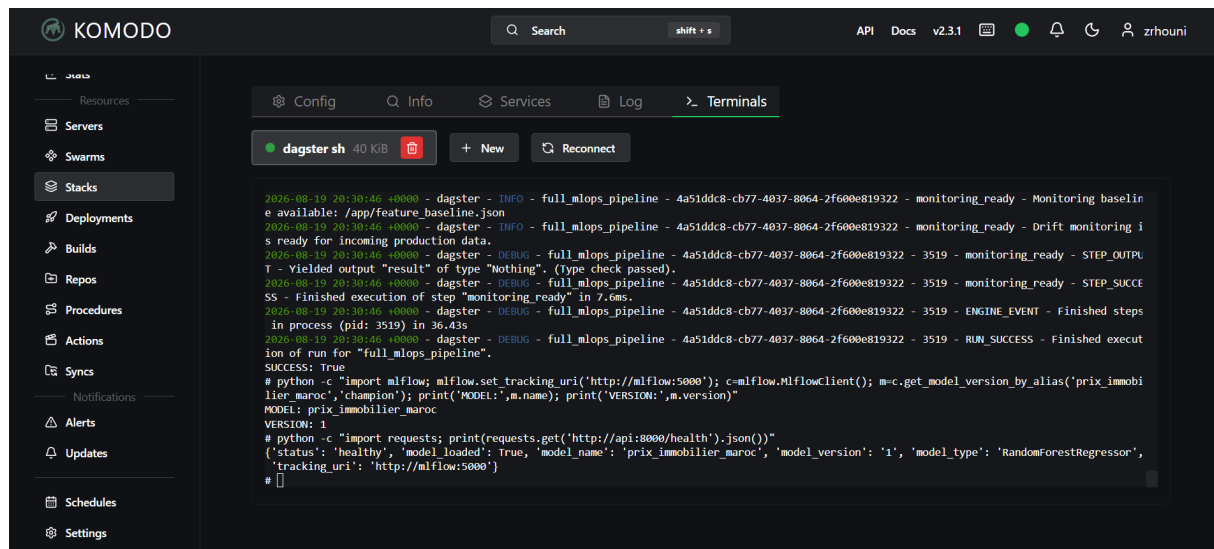


Figure 16.2: Services Komodo à l'état RUNNING

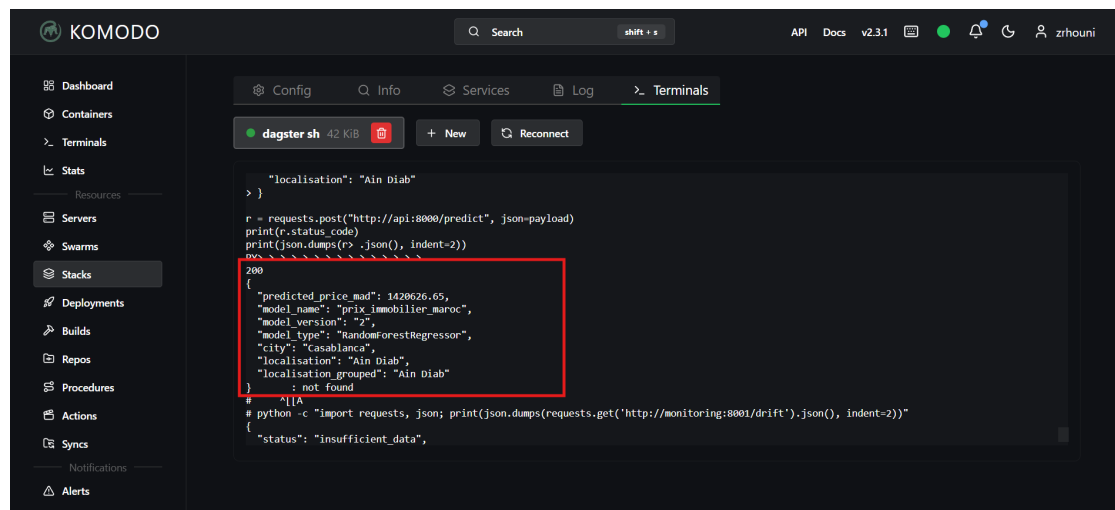


Figure 16.3: Exécution réussie du pipeline Dagster

CHAPTER 17

Ajout du monitoring persistant

17.1 Pourquoi ajouter un service de monitoring dédié ?

Un service FastAPI léger dédié au suivi de production a été ajouté, avec trois endpoints : `GET /health`, `GET /metrics` et `GET /drift`, sondant l'API périodiquement et écrivant ses observations dans `availability.jsonl`.

17.2 Validation d'une prédiction après intégration du monitoring

L'API a répondu HTTP 200 avec un prix prédit d'environ **1 420 626,65 MAD** pour un appartement à Casablanca (Ain Diab, 100 m², 2 chambres, 1 salle de bain, étage 2, 3 pièces), en s'appuyant sur la version 2 du modèle `RandomForestRegressor`.

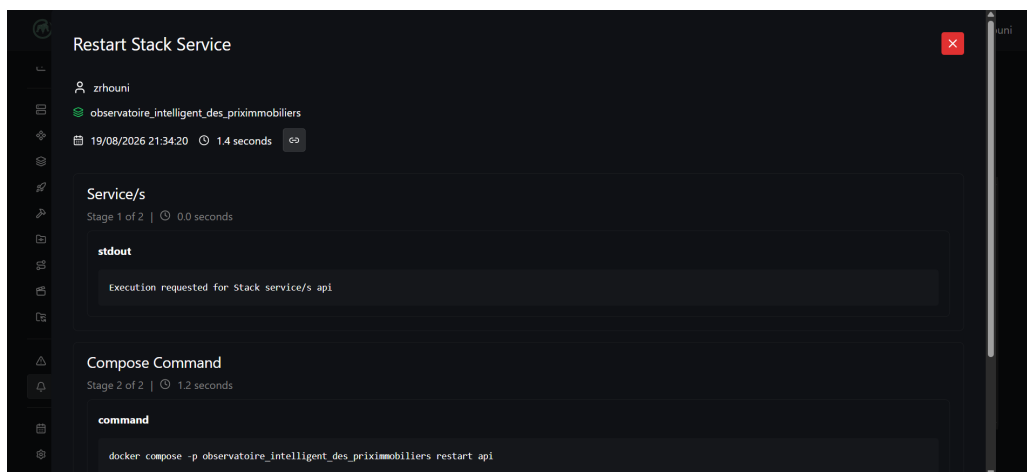


Figure 17.1: Test de prédiction après intégration du monitoring

17.3 Test de persistance après redémarrage de l'API

Le service API a été explicitement redémarré depuis Komodo, prouvant que l'API récupère bien son état à partir de ressources persistantes plutôt qu'à partir d'une initialisation volatile.

17.4 Validation de la dérive avec garde-fou sur le nombre d'échantillons

Un seuil `DRIFT_MIN_SAMPLES=20` a été ajouté. Au moment de la validation finale, le monitoring indiquait `samples_analyzed = 1`, `minimum_required = 20` et `drift_detected = false`, un comportement conforme à ce qui était attendu.

CHAPTER 18

Erreurs rencontrées et solutions appliquées

primaryblue!15 N°	Problème observé	Cause principale	Correction réalisée
1	Chemins DuckDB incohérents entre scripts	Chemins relatifs ou codés en dur	Remplacement par <code>DUCKDB_PATH</code> et harmonisation avec <code>/runtime/data</code>
2	Entraînement local / conteneur ne lisant pas la même base	Composants non pilotés par les mêmes variables d'environnement	Uniformisation via <code>os.getenv(...)</code>
3	Import du module d'entraînement déclenchant une configuration MLflow trop tôt	Initialisation au chargement du module	Déplacement de la configuration MLflow dans le flux principal
4	Alias du modèle champion non géré explicitement	Enregistrement sans affectation fiable d'un alias	Alias champion assigné manuellement au modèle enregistré
5	API couplée à une version précise du modèle	Utilisation directe d'un numéro de version, fragile	Chargement via <code>REGISTERED_MODEL_NAME + MODEL_ALIAS</code>
6	Services Komodo restant DOWN après mise à jour	La configuration seule ne déclenche pas le démarrage	Déclenchement explicite du Deploy Stack
7	Démon Dagster ne partageant pas l'état du webserver	Absence de volume commun	Volume <code>dagster_data</code> commun et <code>DAGSTER_HOME</code> partagé
8	API devant rester disponible sans modèle enregistré	Échec potentiel au démarrage	Comportement <code>waiting_for_model</code>
9	Notebooks historiques faisant échouer Ruff	Périmètre Ruff trop large	Exclusion des notebooks de la commande Ruff en CI
10	Baseline de dérive risquant de disparaître après redéploiement	Fichier produit dans le système de fichiers interne	Volume <code>monitoring_data</code>
11	Prédictions à observer sans casser l'API en cas d'échec disque	Un échec de journalisation ne doit pas annuler l'inférence	Journalisation <code>warning-only</code>
12	Monitoring déclarant une dérive sur trop peu de données	PSI non interprétable sur peu d'observations	Seuil <code>DRIFT_MIN_SAMPLES=20</code>

primaryblue!15 N°	Problème observé	Cause principale	Correction réalisée
13	Accès externe aux ports limité malgré fonctionnement interne correct	Règles réseau du serveur hors contrôle du projet	Validation interne par appels inter-services
14	Rejet du push final sur la branche distante	Évolution de la branche entre-temps	Synchronisation par <code>git fetch</code> puis <code>git rebase</code>

Table 18.1: Principales erreurs rencontrées et solutions appliquées

CHAPTER 19

Résultats finaux et synthèse de conformité

19.1 Résultats fonctionnels

À la fin du travail, les éléments suivants étaient validés : reconstruction locale de la stack ; ingestion dlt vers DuckDB persistante ; transformations dbt exécutées ; qualité des données validée ; entraînement et enregistrement du modèle dans MLflow ; mise à jour de l'alias champion ; API FastAPI fonctionnelle ; pipeline Dagster exécutable sur le serveur ; déploiement Komodo opérationnel ; monitoring disponibilité / latence / métriques ML / drift fonctionnel.

19.2 Synthèse de conformité

primaryblue!15 Exigence	État	Preuve / implémentation
DAG dlt → dbt → tests → ML → monitoring	OK	Job <code>full_mlops_pipeline</code> exécuté avec succès sur le serveur
FastAPI /predict et /health	OK	HTTP 200, modèle chargé et prédiction validée
Dockerfile + Docker Compose	OK	Stack reconstruite localement puis déployée sur Komodo
CI tests / lint / build	OK	GitHub Actions verte
Tracking et Registry MLflow	OK	Modèle <code>prix_immobilier_maroc</code> , versions et alias champion
Qualité dbt	OK	19/19 tests PASS
Disponibilité service	OK	Sonde périodique et journal <code>availability.jsonl</code>
Temps de réponse	OK	Latence courante, moyenne et p95 exposées par <code>/metrics</code>
Métriques ML	OK	RMSE, MAE et R^2 lus dynamiquement depuis MLflow
Drift simple	OK	PSI par feature avec garde-fou <code>DRIFT_MIN_SAMPLES=20</code>
Persistance monitoring	OK	Volume <code>monitoring_data</code>

Table 19.1: Matrice de conformité des livrables techniques

19.3 Bonnes pratiques retenues

- Configuration par variables d'environnement plutôt que chemins codés en dur.
- Volumes persistants pour les données et états critiques.
- Alias MLflow pour découpler l'API d'une version numérique.

- Quality gates dbt avant l'entraînement (héritées de la Partie II).
- API résiliente lorsqu'aucun modèle n'est encore enregistré.
- Validation locale systématique avant passage au serveur.
- Déploiement multi-services cohérent via Docker Compose.
- Monitoring persistant et garde-fou sur le nombre minimal d'observations.

Conclusion générale

Le travail présenté dans ce rapport ne s'est pas limité à faire fonctionner un service isolé. Il retrace une chaîne cohérente allant de l'organisation Agile de l'équipe (Partie I) à la garantie de la qualité des données consommées par les modèles (Partie II), jusqu'à la reconstruction, l'intégration, la correction puis la stabilisation de tout le pipeline DataOps / MLOps (Partie III).

La démarche a volontairement commencé par le cadrage et la planification Scrum, qui a structuré le travail de huit membres sur quatre sprints et 95 story points, tous intégralement réalisés sans report. C'est dans ce cadre que les incohérences de chemins, de dépendances et de configuration sont apparues et ont pu être anticipées via le registre des risques et le registre des obstacles. Une fois la qualité des données garantie par des Data Contracts stricts, il a été possible de rendre la chaîne cohérente, de l'orchestrer avec Dagster, de l'exposer avec FastAPI, de la suivre avec MLflow, de la valider en CI, puis de la déployer sur Komodo.

L'ajout final d'un monitoring persistant a permis de compléter l'industrialisation en couvrant la disponibilité, la latence, les métriques ML et la dérive. Le résultat final est une solution cohérente, testée, conteneurisée et démontrable, qui répond de façon solide aux exigences du module DataOps et MLOps, en s'appuyant de bout en bout sur dlt, DuckDB, dbt, Dagster, scikit-learn (Random Forest Regressor), MLflow, FastAPI, Docker Compose, GitHub Actions, Komodo et un service de monitoring dédié :

Collecter → Fiabiliser → Apprendre → Industrialiser